

Plano de Trabalho

1. Identificação

Este Plano de Trabalho descreve a proposta para o presente projeto de trabalho de conclusão de curso (TCC) que será executado pelos alunos da segunda turma do programa CI Digital no polo Inatel, conforme descritos na tabela de identificação a seguir:

Título do Projeto	Arquitetura em FPGA para extração de features de fluxos TCP/IP visando sistemas de detecção de intrusão em rede baseados em Machine Learning
Duração de Execução	6 meses
Mês de Início	Setembro de 2026
Mês de Fim	Fevereiro de 2027
Tópico Principal	Projeto de Circuitos Digitais
Subáreas	Circuitos Digitais, Cibersegurança, Detecção de ameaças, Sistema de detecção de intrusão, Aprendizado de máquina, Redes TCP/IP, Redes IoT
Membros	Alessandra Carolina Domiciano Beatriz Bastos Assis Bruno Augusto Caetano Coura Daniela Maria Barbosa Faria Elaine Cristina De Cássia Silva Fábio Henrique Moreira Herman Cristiano Jaime Samuel Josias Ross
Orientador	Dr. Elivander Judas Tadeu Pereira

2. Objeto

2.1. Resumo

Este projeto propõe o desenvolvimento de uma arquitetura em FPGA para a extração de características baseadas em fluxo a partir de tráfego TCP/IP, com o objetivo de produzir vetores de features que possam ser posteriormente utilizados como entrada para Sistemas de Detecção de Intrusão de Rede (NIDS) baseados em aprendizado de máquina. A arquitetura deverá processar pacotes Ethernet II, IPv4, TCP e UDP, identificar fluxos unidirecionais, manter uma tabela de fluxos ativos e atualizar continuamente as informações associadas a cada comunicação.

Considerando que diferentes datasets e ferramentas de extração adotam conjuntos distintos de features, o projeto também contempla a definição de um subconjunto de características relevante para aplicações de detecção de intrusões e viável para implementação em hardware. A primeira prova de conceito será baseada nas features `packet_count`, `byte_count` e `duration`, podendo ser posteriormente ampliada com outras informações estatísticas, temporais e de cabeçalho, como tamanhos de pacotes, intervalos entre chegadas, taxas e flags TCP.

O desenvolvimento será validado inicialmente em simulação, utilizando tráfego sintético e pacotes provenientes de arquivos PCAP, com comparação dos resultados obtidos com um modelo de referência em software. Também será utilizado um dataset benchmark de ameaças para verificar o funcionamento da arquitetura com tráfego representativo. Por fim, a solução será implementada em FPGA para avaliação de sua viabilidade, considerando aspectos como utilização de recursos, frequência de operação, latência e capacidade de processamento. O classificador de aprendizado de máquina não faz parte do escopo desta etapa do projeto.

2.2. Objetivo Geral

Desenvolver uma arquitetura em HDL para FPGA capaz de processar continuamente tráfego TCP/IP e realizar a extração de características baseadas em fluxo, gerando vetores de features destinados à utilização por NIDS baseados em aprendizado de máquina.

2.3. Objetivos Específicos

São objetivos para a execução deste Plano de Trabalho:

- i. Definir um conjunto reduzido de características de tráfego baseadas em fluxo, considerando sua relevância para NIDS baseados em aprendizado de máquina e sua viabilidade de obtenção em hardware.
- ii. Implementar o processamento dos pacotes para extração das informações das camadas de rede e transporte, contemplando inicialmente Ethernet II, IPv4, TCP e UDP e produzindo os metadados necessários à identificação dos fluxos.
- iii. Implementar a identificação dos fluxos a partir da 5-tupla, permitindo associar cada pacote à comunicação correspondente e gerar a chave e o índice necessários ao acesso às estruturas de armazenamento.
- iv. Implementar uma tabela de fluxos capaz de armazenar e atualizar o estado associado a cada fluxo identificado, mantendo as informações necessárias à extração incremental das características selecionadas.

- v. Implementar a extração das características de fluxo selecionadas, contemplando inicialmente contagem de pacotes, volume de dados e duração do fluxo, de forma integrada ao processamento dos pacotes.
- vi. Desenvolver e aplicar uma estratégia de verificação funcional utilizando tráfego sintético e proveniente de arquivos PCAP, modelo de referência em software e casos de teste representativos, verificando a correta extração dos metadados, identificação dos fluxos e geração das características esperadas.
- vii. Verificar o funcionamento da arquitetura com tráfego proveniente de um dataset benchmark de ameaças, avaliando sua operação com tráfego representativo e compatível com o escopo da primeira prova de conceito.
- viii. Implementar e validar a arquitetura em FPGA, avaliando a utilização de recursos lógicos e de memória, frequência de operação, latência e capacidade de processamento da solução desenvolvida.

3. Relevância do Projeto

3.1. Problema de Pesquisa e Justificativa

A expansão das redes de computadores e da Internet das Coisas (IoT) tem ampliado de forma expressiva a quantidade de dispositivos conectados, ao mesmo tempo em que contribui para o aumento da superfície de ataque e da exposição de sistemas e usuários a ameaças cibernéticas. Nesse contexto, os Sistemas de Detecção de Intrusão de Rede (NIDS) têm como finalidade monitorar o tráfego em busca de indícios de atividade maliciosa na rede. Entre as abordagens utilizadas para essa tarefa, técnicas de aprendizado de máquina (ML) têm recebido atenção por permitirem que padrões presentes no tráfego sejam explorados na distinção entre comunicações legítimas e atividades potencialmente maliciosas.

A eficácia desse tipo de sistema depende não apenas do algoritmo de aprendizado empregado, mas também da forma como o tráfego é representado para o modelo, por meio de um conjunto de características (*features*) extraídas do tráfego de rede. Nesse sentido, a análise baseada em fluxos (*flow-based*) apresenta vantagens por permitir a caracterização do tráfego a partir de informações agregadas dos fluxos, sem depender da inspeção do *payload*, cujo conteúdo pode estar protegido por criptografia [1]. Estudos demonstram, por exemplo, que características temporais associadas aos fluxos podem ser utilizadas para identificar tráfego VPN e representar diferentes tipos de tráfego criptografado [2].

Para disponibilizar as *features* de fluxo ao classificador, é necessário realizar sua extração a partir dos pacotes que compõem cada comunicação. Essa etapa consiste em identificar os fluxos e transformar informações observadas no tráfego em atributos representativos, como contagens, volumes, duração e outras estatísticas. Dessa forma, a definição de quais características serão extraídas possui influência direta sobre a representação do tráfego fornecida ao modelo de detecção.

A escolha das *features* tem sido amplamente investigada na literatura. As ferramentas e datasets empregados em trabalhos de detecção de intrusão em redes apresentam conjuntos bastante distintos de atributos, dificultando a definição de um conjunto comum de características [3]. Além disso, a utilização de um número maior de atributos não implica necessariamente melhor desempenho na detecção, uma vez que informações irrelevantes ou redundantes podem aumentar o custo computacional e até prejudicar a capacidade de classificação [4]. Por outro lado, a eliminação de características pouco informativas pode reduzir significativamente o conjunto de atributos sem comprometer o desempenho de detecção [5]. A literatura também indica que a relevância das *features* pode variar de acordo com o tipo de ataque, de modo que conjuntos reduzidos selecionados para classes específicas de ameaça podem apresentar vantagens em relação à aplicação da mesma seleção a todo o dataset [6]. Dessa forma, a seleção de um conjunto adequado de características permanece como um desafio no desenvolvimento de NIDS baseados em ML.

Além da escolha das *features* que irão representar o tráfego, a própria etapa de extração impõe requisitos de desempenho ao sistema. Para que as informações de fluxo possam ser disponibilizadas em tempo adequado ao mecanismo de detecção, esse processamento precisa acompanhar a taxa de tráfego da rede. Como a análise *flow-based* possui natureza *stateful*, é necessário manter informações associadas a cada fluxo ativo e atualizá-las à medida que novos pacotes são recebidos. Isso implica identificar o fluxo correspondente, recuperar seu estado, modificar contadores, *timestamps* e outras estatísticas e armazenar novamente essas informações. Em cenários de alta velocidade, soluções baseadas em software podem

apresentar limitações, principalmente quando é necessário acompanhar simultaneamente uma grande quantidade de fluxos e realizar essas atualizações continuamente [7].

Nesse cenário, a implementação em hardware surge como uma alternativa para acelerar essa etapa de processamento. FPGAs apresentam características particularmente adequadas ao processamento de tráfego de rede, como paralelismo, organização em *pipeline* e processamento em *streaming*, permitindo que os dados sejam tratados à medida que os pacotes são recebidos. Essas propriedades favorecem a construção de arquiteturas dedicadas capazes de acompanhar altas taxas de transmissão e executar diferentes etapas do processamento de rede de forma contínua.

A possibilidade de implementação de NIDS em hardware tem sido explorada em diferentes etapas do processamento de tráfego de rede em FPGA. Um estudo sistemático das principais abordagens de NIDS implementadas nesse tipo de dispositivo inclui técnicas de *pattern matching*, monitoramento de fluxos e aprendizado de máquina, evidenciando o emprego de hardware reconfigurável em sistemas que precisam atender a requisitos de processamento em tempo real [8].

Além disso, diferentes trabalhos têm explorado estágios específicos do processamento de tráfego de rede em FPGA. Na etapa de *parsing*, arquiteturas modulares organizadas em *pipeline* já demonstraram capacidade de operação em taxas superiores a 100 Gb/s [9]. No processo de identificação e manutenção dos fluxos, também foram propostas arquiteturas baseadas em *Cuckoo Hashing* em *pipeline*, nas quais os pacotes são associados às respectivas entradas de fluxo e suas informações são armazenadas e atualizadas em memória, alcançando processamento de 100 Gb/s sem necessidade de amostragem [7].

No contexto específico da extração de características, arquiteturas FPGA já foram propostas para calcular estatísticas de fluxo de forma *online*, alcançando taxas de até 96 Gb/s e suportando dezenas de milhares de fluxos concorrentes [10]. Lu et al. [11] descrevem em Verilog um mecanismo de rastreamento com tabela de estado para até oito mil fluxos concorrentes, reportando ocupação de 9.051 elementos lógicos e 21,5 blocos de memória embarcada a 125 MHz, voltado à aceleração genérica de inferência sobre tráfego. Outros trabalhos demonstram a aplicação da extração de características em hardware diretamente no contexto de NIDS baseados em aprendizado de máquina, considerando atributos obtidos dos cabeçalhos dos pacotes e características agregadas por host e serviço, com ganhos de desempenho e eficiência energética em relação à implementação em software [12].

Em contraste com as abordagens voltadas ao processamento do tráfego e à extração de características, outros trabalhos concentram a implementação em hardware na etapa posterior de classificação, assumindo que os vetores de entrada já tenham sido previamente produzidos em software. Pham-Quoc, Tran e Thinh [13], por exemplo, implementam em Verilog um sistema de detecção de anomalias em FPGA Virtex-7, ocupando até 21,39% dos elementos lógicos e operando a 206,7 MHz, no qual o esforço de projeto concentra-se no acelerador da rede neural, enquanto as características utilizadas provêm de conjuntos de dados cujos vetores foram previamente computados por ferramentas de software.

De forma geral, os trabalhos analisados apresentam objetivos e recortes distintos, refletindo diferentes formas de organizar e explorar as informações do tráfego. Nesse contexto, o desenvolvimento de um extrator de características envolve dois aspectos complementares. Por um lado, é necessário produzir uma representação do tráfego que preserve informações relevantes para aplicações de detecção. Por outro lado, sua implementação deve considerar

tanto a complexidade das operações necessárias quanto a capacidade de processamento exigida para acompanhar o tráfego em alta velocidade.

Diante disso, este projeto propõe o desenvolvimento de uma arquitetura em FPGA para a extração de features de fluxos de tráfego TCP/IP, visando produzir informações que possam ser posteriormente utilizadas como entrada para NIDS baseados em ML. O trabalho busca avaliar a viabilidade da execução desta etapa em hardware reconfigurável, explorando suas características de processamento dedicado. O escopo concentra-se na extração de características, etapa que antecede a classificação, não abrangendo, portanto, a implementação do modelo de aprendizado de máquina.

3.2. Desafios tecnológicos

A implementação da extração de *features* em FPGA envolve diferentes desafios. A seguir estão descritas algumas das questões a serem enfrentadas nesse projeto:

- **Parsing de pacotes em alta velocidade:** o sistema deve identificar corretamente os campos dos protocolos Ethernet, IPv4 e TCP/UDP sem interromper o fluxo de dados. Como os cabeçalhos possuem tamanhos e posições variáveis, o processamento pode exigir diferentes etapas de interpretação. Isso pode ser feito através de módulos ou estágios de pipeline, permitindo que diferentes partes do pacote sejam processadas de forma sequencial e organizada.
- **Identificação e manutenção dos fluxos:** cada pacote recebido deve ser associado ao fluxo correspondente, e as informações acumuladas desse fluxo precisam ser preservadas entre diferentes pacotes. Para isso, é preciso definir uma chave de identificação de pacotes, como a 5-tupla, associada a uma estrutura de armazenamento capaz de manter o estado de cada fluxo ativo.
- **Busca eficiente na tabela de fluxos:** procurar sequencialmente por um fluxo em uma tabela extensa pode limitar o desempenho do sistema. Para reduzir o tempo de consulta, podem ser utilizadas funções hash que transformam a chave do fluxo em um índice de memória, permitindo acesso mais direto às informações armazenadas.
- **Tratamento de colisões de hash:** diferentes fluxos podem produzir o mesmo índice de acesso à tabela. Dessa forma, informações de fluxos distintos podem ser sobrescritas ou perdidas. Possíveis soluções para isso são o uso de múltiplas funções hash, posições alternativas, tabelas auxiliares, encadeamento ou técnicas como *Cuckoo Hashing*.
- **Dependência entre pacotes do mesmo fluxo:** quando dois ou mais pacotes consecutivos pertencem ao mesmo fluxo, um novo pacote pode acessar o registro de estado antes que a atualização causada pelo pacote anterior tenha sido concluída, gerando inconsistências nos acumuladores. Uma solução simples é utilizar *backpressure* e aguardar a conclusão da atualização anterior. Arquiteturas mais avançadas empregam detecção de conflitos, *forwarding* dos valores atualizados ou memórias temporárias.
- **Atualização incremental das features:** características como quantidade de pacotes, quantidade de bytes, duração e flags precisam ser atualizadas à medida que novos pacotes são recebidos. Features simples podem ser implementadas com contadores, registradores e comparadores, enquanto cálculos como média, variância, desvio-padrão e taxas exigem operações mais complexas. Uma estratégia possível é iniciar com um conjunto reduzido de features e acrescentar características progressivamente conforme a disponibilidade de recursos.

- **Dimensionamento da memória:** cada fluxo ativo necessita de um registro contendo sua chave e suas estatísticas. O aumento do número de fluxos ou de features aumenta diretamente a quantidade de memória necessária. Esse problema pode ser mitigado adotando políticas de remoção e substituição de fluxos.
- **Finalização dos fluxos:** o sistema precisa definir quando um fluxo deixa de ser atualizado e seu vetor de features pode ser gerado. Como nem todos os protocolos possuem uma indicação explícita de término, podem ser utilizados critérios como flags TCP, timeout de inatividade, duração máxima ou número máximo de pacotes. A política adotada influencia diretamente a quantidade de fluxos mantidos simultaneamente em memória.
- **Sincronização entre os módulos:** diferentes etapas da arquitetura podem possuir latências distintas, tornando necessário controlar quando os dados estão disponíveis e quando cada módulo pode aceitar novas informações. Esse problema pode ser tratado com protocolos de handshake, como valid/ready, além de sinais específicos para indicar conclusão de leituras, escritas e atualizações.
- **Equilíbrio entre throughput e complexidade:** permitir o processamento simultâneo de vários pacotes aumenta o desempenho, mas também aumenta a possibilidade de dependências e conflitos de memória. Por outro lado, bloquear a entrada de novos dados durante determinadas operações simplifica o controle, mas reduz o throughput. Uma alternativa é iniciar com uma arquitetura sequencial ou com *backpressure* e posteriormente avaliar a introdução de pipeline e mecanismos de resolução de conflitos.
- **Utilização dos recursos da FPGA:** memória, LUTs, registradores e blocos aritméticos são recursos limitados. O número de fluxos suportados, a quantidade de features e o grau de paralelismo precisam ser definidos considerando a capacidade do dispositivo utilizado. A modularização da arquitetura e a implementação progressiva das funcionalidades facilitam a avaliação desse compromisso.

3.3. Referências

- [1] UMER, Muhammad Fahad; SHER, Muhammad; BI, Yaxin. Flow-based intrusion detection: techniques and challenges. *Computers & Security*, v. 70, p. 238–254, 2017. DOI: 10.1016/j.cose.2017.05.009.
- [2] DRAPER-GIL, Gerard; HABIBI LASHKARI, Arash; MAMUN, Mohammad Saiful Islam; GHORBANI, Ali A. Characterization of encrypted and VPN traffic using time-related features. In: *Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP)*. Rome, 2016. p. 407–414. DOI: 10.5220/0005740704070414.
- [3] SARHAN, Mohanad; LAYEGHY, Siamak; PORTMANN, Marius. Towards a standard feature set for network intrusion detection system datasets. *arXiv preprint arXiv:2101.11315*, 2021.
- [4] YIN, Yuhua; JANG-JACCARD, Julian; XU, Wen; SINGH, Amardeep; ZHU, Jinting; SABRINA, Fariza; KWAK, Jin. IGRF-RFE: a hybrid feature selection method for MLP-based network intrusion detection on UNSW-NB15 dataset. *arXiv preprint arXiv:2203.16365*, 2022.
- [5] WESTPHAL, Charles; HAILES, Stephen; MUSOLESI, Mirco. Feature selection for network intrusion detection. *arXiv preprint arXiv:2411.11603*, 2024.

- [6] CHOI, Dayoung; RHEEY, Joohong; PARK, Hyunggon. Attack-specific feature analysis framework for NetFlow IoT datasets. *Computers & Security*, v. 157, art. 104536, 2025. DOI: 10.1016/j.cose.2025.104536.
- [7] SHA, Meng; GUO, Zhichuan; WANG, Ke; ZENG, Xuwen. A high-performance and accurate FPGA-based flow monitor for 100 Gbps networks. *Electronics*, v. 11, n. 13, art. 1976, 2022. DOI: 10.3390/electronics11131976.
- [8] LE JEUNE, Laurens; SATEESAN, Arish; RABBANI, Md Masoom; GOEDEMÉ, Toon; VLIEGEN, Jo; MENTENS, Nele. SoK – Network intrusion detection on FPGA. In: *Security, Privacy, and Applied Cryptography Engineering*. Lecture Notes in Computer Science, v. 13162. Cham: Springer, 2022. p. 242–261. DOI: 10.1007/978-3-030-95085-9_13.
- [9] KEKELY, Lukáš; PUŠ, Viktor; KOŘENEK, Jan. Low-latency modular packet header parser for FPGA. In: *ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. Austin: ACM, 2012. p. 77–78. DOI: 10.1145/2396556.2396571.
- [10] TONG, Da; PRASANNA, Viktor K. Dynamically configurable online statistical flow feature extractor on FPGA. In: *2013 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2013. p. 1–6. DOI: 10.1109/HPEC.2013.6670346.
- [11] LU, Yingying et al. Octopus: a heterogeneous in-network computing accelerator enabling deep learning for network. arXiv:2308.11312, 2023.
- [12] VIEGAS, Eduardo; SANTIN, Altair O.; FRANÇA, André; JASINSKI, Ricardo; PEDRONI, Volnei A.; OLIVEIRA, Luiz S. Towards an energy-efficient anomaly-based intrusion detection engine for embedded systems. *IEEE Transactions on Computers*, v. 66, n. 1, p. 163–177, 2017. DOI: 10.1109/TC.2016.2560839.
- [13] PHAM-QUOC, Cuong; TRAN, Nguyen Tho Huy; THINH, Tran Ngoc. FPGA/AI-powered architecture for anomaly network intrusion detection systems. **Electronics**, v. 12, n. 3, art. 668, 2023. DOI: 10.3390/electronics12030668.
- [14] LE JEUNE, Laurens; GOEDEMÉ, Toon; MENTENS, Nele. Towards real-time deep learning-based network intrusion detection on FPGA. In: *Applied Cryptography and Network Security Workshops (ACNS 2021)*. Lecture Notes in Computer Science, v. 12809. Cham: Springer, 2021. p. 133–150. DOI: 10.1007/978-3-030-81645-2_9.
- [15] ROSAY, Arnaud; CHEVAL, Eloïse; CARLIER, Florent; LEROUX, Pascal. From CIC-IDS2017 to LYCOS-IDS2017: a corrected dataset for better performance. In: *Proc. IEEE/WIC/ACM International Conference on Web Intelligence*, 2021. DOI: 10.1145/3486622.3493973.

4. Metodologia

4.1. Arquitetura

A arquitetura do projeto será estruturada em dois blocos, que executam as duas etapas principais de processamento do sistema: Extração de Metadados e Extração de Features. O primeiro é responsável por receber os pacotes TCP/IP e gerar os metadados de cada um, além da chave de identificação e do índice hash correspondentes ao fluxo ao qual cada pacote pertence. Essas informações são fornecidas posteriormente ao bloco Extração de Features, para geração do vetor de features baseado em fluxo.

A **Figura 1** apresenta uma visão de alto nível da arquitetura proposta.

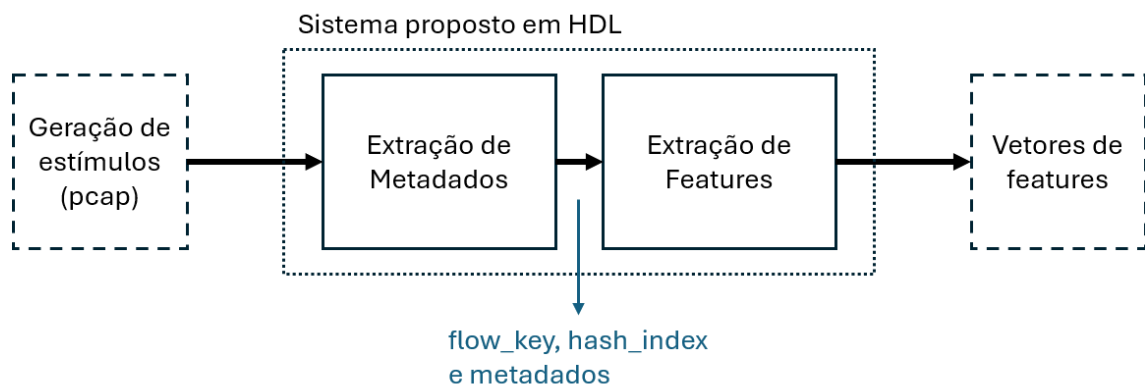


Figura 1. Representação da arquitetura.

i. Extração de Metadados:

O bloco de Extração de Metadados será responsável por transformar os pacotes recebidos pela interface de entrada em informações que permitam identificar o fluxo ao qual cada pacote pertence, determinar o índice de consulta na Flow Table e disponibilizar os metadados necessários para o processamento das features de cada fluxo. Sua arquitetura será organizada em três módulos principais: Packet Parser, Flow Identification Unit e Hash Unit.

a. Packet Parser

O Packet Parser será responsável por interpretar os pacotes TCP/IP recebidos pela interface de entrada e extrair os campos dos cabeçalhos necessários às etapas seguintes. Em um primeiro momento serão considerados pacotes contendo Ethernet, IPv4 e TCP ou UDP. Casos como ARP, IPv6, VLAN, ICMP, IPv4 com opções e fragmentação não fazem parte do escopo inicial.

Para cada pacote válido, o parser deverá disponibilizar, no mínimo:

- endereço IP de origem (src_ip);
- endereço IP de destino (dst_ip);
- porta de origem (src_port);
- porta de destino (dst_port);
- protocolo de transporte (protocol);
- comprimento do pacote (packet_length);
- timestamp associado ao pacote.

A lógica do parser será implementada por meio de uma máquina de estados finitos (FSM) responsável por acompanhar a posição do processamento dentro do pacote. De forma simplificada, poderão ser utilizados estados correspondentes à recepção dos cabeçalhos Ethernet, IPv4 e TCP/UDP, além de estados de espera e finalização. A transição entre os estados ocorrerá conforme os bytes do pacote forem recebidos e os campos necessários forem identificados.

O processamento deverá respeitar o protocolo valid/ready da interface de entrada, de forma que o estado interno e a posição de leitura do pacote avancem somente quando ocorrer uma transferência válida. Ao término da extração dos campos necessários, os metadados serão disponibilizados ao módulo Flow Identification.

b. Flow Identification Unit

O módulo Flow Identification Unit será responsável por gerar a chave utilizada para identificar o fluxo ao qual um pacote pertence.

Nesse momento, será realizada identificação baseada em fluxo unidirecional (uniflow). Dessa forma, um pacote de resposta com endereços IP e portas invertidos deverá gerar uma chave diferente daquela produzida pelo pacote no sentido original.

A chave será formada pela concatenação da 5-tuple: $flow_key = src_ip || dst_ip || src_port || dst_port || protocol$. Considerando endereços IPv4, portas TCP/UDP e o campo de protocolo, a chave possuirá 104 bits.

Para cada conjunto válido de metadados recebido do Packet Parser, o módulo deverá gerar uma $flow_key$ e encaminhá-la à Hash Unit. Os demais metadados, como $packet_length$ e $timestamp$, também deverão ser transferidos para serem utilizados posteriormente pelo bloco de Extração de Features.

c. Hash Unit

A Hash Unit receberá a $flow_key$ de 104 bits e aplicará uma função de hash com o objetivo de gerar o índice utilizado para a consulta inicial da Flow Table.

Para uma mesma $flow_key$, a unidade deverá sempre produzir o mesmo resultado. A saída do módulo também será a saída do bloco Extração de Metadados e conterá o $hash_index$, a $flow_key$ e os demais metadados de pacote, que serão encaminhados ao bloco Extração de Features.

O algoritmo de hash e seus parâmetros serão definidos durante o detalhamento da implementação, assim como a largura do índice de saída.

ii. Extração de Features:

O bloco de Extração de Features será responsável pelo gerenciamento dos fluxos ativos em uma tabela, mantendo e atualizando as informações de seus estados, e pela geração do vetor de features por fluxo disponibilizado na saída do sistema. Sua arquitetura será estruturada em quatro módulos: Controller, Flow Table, Feature Update Unit e Flow Finalizer.

a. Controller

O módulo Controller será responsável pela coordenação de todas as funções dessa etapa, desde a recepção e processamento dos metadados até a finalização dos fluxos e geração das features.

A entrada do módulo consiste na saída do bloco Extração de Metadados. A partir do hash_index, o controlador solicitará a leitura da posição correspondente na Flow Table e utilizará os campos valid e flow_key do registro retornado para classificar a consulta como HIT, MISS ou COLLISION.

- Em caso de MISS, o controlador inicializará uma nova entrada com as informações associadas ao primeiro pacote do fluxo e solicitará sua inserção na tabela.
- Em caso de HIT, encaminhará ao Feature Update Unit o estado atual do fluxo juntamente com os metadados do novo pacote, aguardando a sinalização de conclusão da atualização.
- Em caso de COLLISION, na primeira prova de conceito, o novo fluxo não será inserido na tabela, o pacote correspondente será descartado e a ocorrência será sinalizada.

O módulo também será responsável pelo protocolo valid/ready, pela aplicação de backpressure durante operações em andamento e pelo controle do processo de finalização dos fluxos.

b. Flow Table

A Flow Table será responsável pelo armazenamento do estado dos fluxos monitorados.

Cada entrada conterá, inicialmente, um campo de validade, a chave do fluxo e os valores acumulados necessários à extração das features, incluindo packet_count, byte_count, first_timestamp e last_timestamp.

A tabela será acessada por meio do hash_index e disponibilizará operações de leitura e escrita sobre seus registros. Uma operação de leitura retornará o registro completo associado à posição consultada, permitindo que o Controller determine o resultado da consulta e encaminhe o estado do fluxo aos demais módulos quando necessário.

Na implementação inicial, a tabela será dimensionada com um número reduzido de entradas, facilitando a simulação e a verificação funcional. Posteriormente, sua capacidade será ampliada e sua implementação será adaptada para utilização eficiente dos recursos de memória interna da FPGA.

c. Feature Update Unit

O módulo Feature Update Unit será responsável pela atualização do estado de um fluxo ativo na Flow Table.

Quando o Controller identificar um HIT, o módulo receberá o registro atual do fluxo, o índice correspondente na tabela e os metadados associados ao novo pacote. A partir dessas informações, serão atualizados os valores acumulados, inicialmente, com as seguintes operações:

- $\text{packet_count} = \text{packet_count} + 1$
- $\text{byte_count} = \text{byte_count} + \text{packet_length}$
- $\text{first_timestamp} = \text{first_timestamp}$
- $\text{last_timestamp} = \text{timestamp}$

Após os cálculos, o estado atualizado será escrito novamente na posição correspondente da Flow Table. Ao término da operação, o módulo sinalizará sua conclusão ao Controller.

d. Flow Finalizer

O Flow Finalizer será responsável pela geração do vetor final de features a partir do estado acumulado de cada fluxo.

A finalização será realizada ao término do arquivo PCAP. Nesse momento, o Controller percorrerá sequencialmente as posições da Flow Table e encaminhará o registro completo do fluxo de cada entrada válida ao Flow Finalizer.

A partir dos valores acumulados, o módulo calculará as informações que dependem do estado completo do fluxo, como a duração, e organizará os valores resultantes no vetor de features disponibilizado na saída do sistema.

Após a geração do vetor, o Flow Finalizer sinalizará a conclusão ao Flow Table Controller, permitindo que a entrada correspondente seja disponibilizada.

4.2. Plano de Atividades

O plano de execução do projeto será estruturado em quatro atividades principais. Duas delas correspondem ao desenvolvimento dos dois blocos principais da arquitetura, Extração de Metadados e Extração de Features. Há uma terceira destinada ao desenvolvimento do Ambiente de Verificação e, por fim, haverá uma atividade transversal de Documentação.

A seguir são apresentadas as principais atividades previstas em cada frente.

i. Extração de Metadados

- Especificação dos módulos do bloco Extração de Metadados: Packet Parser, Flow Identification Unit e Hash Unit;
- Especificação dos protocolos suportados e do algoritmo de Hash;
- Implementação dos módulos do bloco com validação individual;
- Integração e verificação funcional do bloco, com validação do caminho completo desde a recepção do pacote até a geração correta dos metadados.

ii. Extração de Features

- Especificação dos módulos do bloco Extração de Features: Controller, Flow Table, Feature Update Unit e Flow Finalizer;
- Especificação das políticas de tratamento de colisão e de finalização de fluxo;
- Implementação dos módulos do bloco com validação individual;
- Integração e verificação funcional do bloco, com validação do gerenciamento dos fluxos e da geração final do vetor de features.

iii. Ambiente de Verificação

- Especificação das interfaces de entrada e saída e do formato dos arquivos de estímulos e resultados;

- Geração de dados sintéticos para testes controlados;
- Desenvolvimento do conversor de arquivos PCAP para o formato de estímulos utilizado em simulação;
- Implementação de testbenches convencionais para verificação dos blocos intermediários e da arquitetura completa;
- Definição do método de captura e associação das saídas do sistema;
- Implementação de um modelo de referência em software para geração dos resultados esperados;
- Definição e execução de casos de teste com tráfego sintético;
- Seleção e preparação de um dataset benchmark de ameaças;
- Verificação do sistema completo com tráfego representativo proveniente do dataset benchmark;
- Implementação e validação do sistema em FPGA.

iv. Documentação

- Documentação da arquitetura, interfaces e decisões de projeto;
- Registro dos procedimentos e resultados de simulação e verificação;
- Registro dos resultados de síntese e validação em FPGA;
- Elaboração da documentação final e apresentação dos resultados.

4.3. Equipe Executora e Responsabilidades

Nome	Função no Projeto	Alocação de Atividades
Alessandra Carolina Domiciano	Gestão/Design/Verificação	Extração de Features de Fluxo
Beatriz Bastos Assis	Design/Verificação	Ambiente de Verificação
Bruno Augusto Caetano Coura	Design/Verificação	Extração de Metadados
Daniela Maria Barbosa Faria	Design/Verificação	Ambiente de Verificação
Elaine Cristina De Cássia Silva	Design/Verificação	Extração de Features de Fluxo
Fábio Henrique Moreira	Design/Verificação	Extração de Features de Fluxo
Herman Cristiano Jaime	Design/Verificação	Extração de Metadados
Samuel Josias Ross	Design/Verificação	Extração de Metadados

5. Resultados Esperados

5.1. Entregáveis e critério de aceitação

5.1.1. Extração de Metadados - Bruno, Herman e Samuel

Como resultado desta etapa, espera-se entregar uma estrutura funcional capaz de receber os pacotes provenientes da interface de entrada, extrair os campos necessários dos cabeçalhos, identificar o fluxo correspondente e gerar o índice utilizado para consulta da Flow Table. A estrutura é composta pelos módulos Packet Parser, Flow Identification Unit e Hash Unit.

A seguir está a lista de entregáveis esperados para esta etapa, seguida dos respectivos critérios de aceitação:

Atividade	Entregável
1.1	Especificação das interfaces de todos os módulos Critério de aceitação: Entradas, saídas, larguras dos sinais, sinais de controle e protocolo de comunicação entre Packet Parser, Flow Identification Unit e Hash Unit definidos, documentados e compatíveis entre os módulos.
1.2	Especificação da lógica de funcionamento de todos os módulos Critério de aceitação: Sequências de operação, comportamento do protocolo valid/ready, condições de início e término do processamento e sinalização da validade das saídas descritos para todos os módulos.
1.3	Especificação dos protocolos suportados e do algoritmo de hash Critério de aceitação: Campos extraídos dos cabeçalhos Ethernet, IPv4 e TCP/UDP, respectivos offsets e larguras definidos e documentados, assim como os tipos de pacotes considerados válidos e inválidos. Função de hash, parâmetros utilizados e largura do hash_index são definidos e documentados de forma compatível com o número de entradas previsto para a Flow Table.
1.4	Implementação da recepção dos pacotes pelo Parser Critério de aceitação: Os dados do pacote são recebidos corretamente somente quando ocorre uma transferência válida na interface, caracterizada pela ativação simultânea dos sinais valid e ready, respeitando também os sinais de delimitação e indicação de bytes válidos.
1.5	Implementação da FSM com a lógica principal do Parser

	Critério de aceitação: O parser percorre corretamente os estados de sua FSM, sendo capaz de reconhecer e extrair os metadados desejados. Para cada pacote válido, é disponibilizado um conjunto consistente de: src_ip, dst_ip, src_port, dst_port, protocol, packet_length e timestamp, em sua saída.
1.6	Implementação do tratamento dos pacotes fora do escopo pelo Parser Critério de aceitação: Pacotes não suportados pela primeira prova de conceito, como ARP, IPv6, VLAN, ICMP, IPv4 com opções ou fragmentação, não produzem conjuntos válidos de metadados para os módulos seguintes.
1.7	Implementação do módulo Flow Identification Unit Critério de aceitação: Para cada conjunto válido de metadados recebido, o módulo gera na saída uma flow_key de 104 bits formada por src_ip dst_ip src_port dst_port protocol, e a encaminha junto às informações de packet_length e timestamp associadas.
1.8	Implementação do módulo Hash Unit Critério de aceitação: Para cada flow_key válida, a Hash Unit produz um hash_index dentro da faixa de endereçamento definida para a Flow Table.
1.9	Integração e validação do Bloco Extração de Metadados Critério de aceitação: Para uma sequência conhecida de pacotes, os valores produzidos de src_ip, dst_ip, src_port, dst_port, protocol, flow_key, hash_index, packet_length e timestamp correspondem aos valores esperados, e pacotes fora do escopo não geram saídas válidas.

5.1.2. Extração de Features de Fluxo - Alessandra, Elaine e Fábio

Como resultado desta etapa, espera-se entregar uma estrutura funcional capaz de armazenar e gerenciar o estado de fluxos ativos durante o processamento dos pacotes e gerar o vetor de features correspondente a cada fluxo ao término do processo. A estrutura é composta pelos módulos Controller, Flow Table, Feature Update Unit e Flow Finalizer.

A seguir está a lista de entregáveis esperados para esta etapa seguidos dos respectivos critérios de aceitação:

Atividade	Entregável
2.1	Especificação das interfaces de todos os módulos e da estrutura da Flow Table Campos do registro, larguras e número inicial de entradas da tabela definidos e documentados. As features inicialmente implementadas, suas representações, larguras e informações necessárias ao seu cálculo também são definidas e documentadas.
2.2	Especificação da lógica de funcionamento e sinais de controle de todos os módulos Critério de aceitação: Sequências de operação, handshakes valid/ready, requisições de leitura e escrita e sinalizações de conclusão descritos para todos os módulos.
2.3	Implementação do módulo Flow Table Critério de aceitação: Tabela capaz de armazenar registros de fluxo e disponibilizar interfaces funcionais de leitura e escrita em simulação.
2.4	Implementação da recepção dos metadados e solicitação de LOOKUP pelo Controller Critério de aceitação: Diante de metadados válidos, o Controller aceita o pacote e solicita a leitura da posição correspondente ao hash_index.
2.5	Implementação da classificação do resultado do LOOKUP em HIT, MISS e COLLISION Critério de aceitação: Após a leitura do registro correspondente ao hash_index, o Controller classifica corretamente a consulta como MISS quando valid = 0, HIT quando valid = 1 e a flow_key armazenada corresponde à flow_key recebida, ou COLLISION quando valid = 1 e as chaves são diferentes.
2.6	Implementação do tratamento de MISS e COLLISION Critério de aceitação: Em um MISS, o Controller inicializa corretamente o registro com packet_count = 1, byte_count = packet_length, first_timestamp = timestamp e last_timestamp = timestamp e solicita sua inserção na tabela.

	Em uma colisão, o novo fluxo não é inserido na tabela e a ocorrência é corretamente sinalizada.
2.7	Implementação do tratamento de HIT Critério de aceitação: Em um HIT, o Controller encaminha corretamente o estado atual do fluxo e os metadados do novo pacote ao Feature Update Unit e solicita sua atualização.
2.8	Implementação do módulo Feature Update Unit Critério de aceitação: Para um HIT, os novos valores de packet_count, byte_count e last_timestamp são calculados corretamente, mantendo first_timestamp inalterado.
2.9	Implementação do controle de fluxo e backpressure Critério de aceitação: O Controller controla corretamente o recebimento de novos metadados por meio do protocolo valid/ready, aceitando um novo pacote somente quando estiver disponível para iniciar seu processamento. Durante operações pendentes de leitura, inserção ou atualização, o processamento do pacote atual é mantido até sua conclusão e o próximo pacote não é aceito antes da sinalização correspondente de término, evitando inconsistências na atualização do estado dos fluxos.
2.10	Integração dos módulos Controller e Flow Table para solicitações de LOOKUP e INSERT Critério de aceitação: O Controller consegue solicitar leituras, receber o registro pedido e inserir corretamente novos registros em casos de MISS.
2.11	Integração dos módulos Controller, Feature Update Unit e Flow Table para solicitações de UPDATE Critério de aceitação: Em uma situação de HIT, o Controller solicita a atualização e o Feature Update Unit recebe corretamente o estado e os metadados necessários. O estado calculado pelo Feature Update Unit é escrito corretamente na posição correspondente da Flow Table e a conclusão da operação é sinalizada ao Controller.
2.12	Implementação do módulo Flow Finalizer

	Critério de aceitação: Para um registro válido, o módulo calcula corretamente $\text{duration} = \text{last_timestamp} - \text{first_timestamp}$ e produz o vetor de features especificado.
2.13	Integração dos módulos Controller e Flow Finalizer Critério de aceitação: Ao receber a solicitação de finalização do processamento, Controller interrompe temporariamente a aceitação de novos pacotes e percorre as posições da Flow Table. Os registros válidos são encaminhados ao Flow Finalizer, que produz os respectivos vetores de features e sinaliza a conclusão das operações.
2.14	Integração e validação do Bloco Extração de Features Critério de aceitação: Para sequências conhecidas de pacotes, os registros armazenados e os vetores finais de features correspondem aos valores esperados de <code>packet_count</code> , <code>byte_count</code> e <code>duration</code> .

5.1.3. Ambiente de verificação - Beatriz e Daniela

Como resultado desta etapa, espera-se entregar um ambiente de verificação capaz de gerar estímulos a partir de arquivos PCAP e tráfego sintético, aplicá-los aos módulos desenvolvidos, capturar os resultados produzidos pelo sistema e compará-los com valores esperados obtidos por um modelo de referência em software. A verificação será realizada inicialmente de forma parcial sobre os blocos de Extração de Metadados e Extração de Features e, posteriormente, sobre a arquitetura completa, tanto em simulação quanto na implementação em FPGA.

A seguir está a lista de entregáveis esperados para esta etapa, seguida dos respectivos critérios de aceitação:

Atividade	Entregável
3.1	Especificação da estratégia de verificação Critério de aceitação: Fluxograma da verificação documentado, contemplando arquivo PCAP, conversor Python, arquivo de estímulos, testbench/DUT, captura das saídas e comparação com resultados de referência. Os pontos de observação e os níveis de verificação previstos devem estar definidos.
3.2	Especificação das interfaces de entrada e saída do ambiente de simulação

	Critério de aceitação: Sinais, larguras, protocolo de transferência e comportamento das interfaces utilizadas para aplicação dos pacotes e obtenção dos resultados do DUT definidos e documentados. A necessidade de utilização ou adaptação para AXI-Stream é avaliada considerando a interface de um IP MAC de referência e a futura integração com a FPGA.
3.3	Especificação do formato de estímulos Critério de aceitação: Formato do arquivo gerado a partir do PCAP definido, incluindo representação dos dados do pacote e informações auxiliares, como delimitação dos pacotes, indicação dos bytes válidos e timestamp.
3.4	Geração de pacotes sintéticos Critério de aceitação: O mecanismo desenvolvido permite gerar arquivos PCAP contendo pacotes com campos previamente definidos, possibilitando controlar fluxos, tamanhos de pacotes, timestamps e demais características necessárias aos testes.
3.5	Desenvolvimento do conversor de PCAP para arquivo de estímulos em Python Critério de aceitação: O programa lê um arquivo PCAP e produz corretamente o arquivo de estímulos especificado. Para um conjunto conhecido de pacotes, as informações geradas correspondem aos dados do PCAP original.
3.6	Implementação do testbench parcial para Extração de Metadados Critério de aceitação: O testbench inicializa o DUT, lê o arquivo de estímulos e aplica os pacotes respeitando o protocolo de transferência definido, permitindo observar e registrar as saídas produzidas pelo bloco de Extração de Metadados.
3.7	Validação do bloco de Extração de Metadados Critério de aceitação: Para os casos de teste executados, os campos e metadados produzidos pelo bloco correspondem aos valores esperados para os pacotes suportados, enquanto pacotes fora do escopo não produzem saídas válidas, conforme a especificação.
3.8	Implementação do testbench parcial para Extração de Features

	Critério de aceitação: O testbench inicializa o bloco de Extração de Features, aplica sequências conhecidas de flow_key, hash_index, packet_length e timestamp e permite observar as operações de LOOKUP, INSERT, UPDATE e finalização.
3.9	Validação do bloco de Extração de Features Critério de aceitação: Para sequências conhecidas de pacotes, os estados armazenados na Flow Table e os vetores finais produzidos correspondem aos valores esperados de packet_count, byte_count e duration. Os casos de HIT, MISS e COLLISION são tratados conforme a especificação.
3.10	Definição e criação dos casos de teste com pacotes sintéticos Critério de aceitação: São definidos e gerados casos de teste sintéticos representativos, incluindo fluxos com um único pacote e múltiplos pacotes, fluxos intercalados, pacotes consecutivos do mesmo fluxo, diferentes comprimentos, pacotes fora do escopo e situações de colisão ou descarte. Quando aplicável, os casos são representados por arquivos PCAP sintéticos.
3.11	Definição do método de associação das saídas do sistema Critério de aceitação: É definido e documentado um método que permita capturar os vetores de features produzidos durante a simulação e associá-los de maneira inequívoca aos respectivos fluxos e resultados esperados.
3.12	Implementação do modelo de referência em software Critério de aceitação: O modelo processa os mesmos pacotes utilizados pelo DUT e calcula os valores esperados de packet_count, byte_count e duration para cada fluxo, disponibilizando resultados compatíveis com o método de associação definido.
3.13	Implementação do testbench completo para validação da arquitetura Critério de aceitação: O testbench integra a leitura e aplicação dos estímulos à arquitetura completa, a captura das saídas e os mecanismos necessários para relacionar os resultados produzidos pelo DUT aos valores fornecidos pelo modelo de referência.
3.14	Validação da arquitetura completa

	Critério de aceitação: O fluxo completo, desde a aplicação dos pacotes na interface de entrada até a geração dos vetores de features, é executado corretamente em simulação. Para os casos de teste definidos, os resultados do DUT correspondem aos valores esperados calculados pelo modelo de referência.
3.15	Escolha de um dataset benchmark de ameaças para testes Critério de aceitação: É selecionado um dataset público que disponibilize tráfego benigno e malicioso em formato PCAP, considerando representatividade, compatibilidade com os protocolos suportados e adequação ao escopo da primeira PoC.
3.16	Preparação/adaptação do dataset benchmark para o ambiente de verificação Critério de aceitação: É preparado um subconjunto representativo do dataset, com identificação ou filtragem do tráfego compatível com o escopo do projeto e conversão dos PCAPs selecionados para o formato utilizado pelo ambiente de simulação. Os resultados esperados correspondentes são produzidos pelo modelo de referência.
3.17	Verificação do sistema completo com dataset benchmark Critério de aceitação: O subconjunto preparado do dataset é processado pela arquitetura completa no ambiente de simulação e os vetores de features produzidos pelo DUT são comparados com os resultados calculados pelo modelo de referência.
3.18	Implementação do circuito em FPGA Critério de aceitação: Os módulos são integrados, sintetizados e implementados para a FPGA selecionada, utilizando a interface de comunicação definida para a plataforma, sem erros de implementação. A utilização de recursos e a frequência de operação são registradas.
3.19	Implementação de mecanismo externo para envio de estímulos e captura de resultados em FPGA Critério de aceitação: É implementado um mecanismo para fornecer pacotes ao sistema por meio da interface disponível na FPGA e recuperar externamente os vetores de features produzidos pelo sistema.
3.20	Validação da arquitetura implementada em FPGA

	Critério de aceitação: Pacotes conhecidos e um conjunto representativo do dataset benchmark são processados pela FPGA. Os vetores produzidos correspondem aos resultados obtidos em simulação e pelo modelo de referência. São também registrados os resultados obtidos para latência e capacidade de processamento da arquitetura implementada.
--	---

5.1.4. Documentação

Atividade	Entregável
4.1	Documentação da arquitetura do sistema Critério de aceitação: A arquitetura do sistema contendo os blocos desenvolvidos é descrita e representada por diagramas. As interfaces entre os módulos, as principais decisões de projeto e as limitações da primeira PoC são documentadas de forma consistente com a implementação realizada.
4.2	Documentação dos procedimentos de verificação Critério de aceitação: Os casos de teste, o modelo de referência, os estímulos utilizados e o procedimento de comparação dos resultados estão descritos.
4.3	Registro dos resultados de simulação Critério de aceitação: Os principais resultados obtidos nos testes dos módulos, subsistemas e sistema integrado estão registrados, incluindo eventuais limitações identificadas.
4.4	Elaboração do relatório final do projeto Critério de aceitação: O documento final apresenta de forma consolidada a metodologia, arquitetura, implementação, verificação, resultados, conclusões e trabalhos futuros do projeto.
4.5	Elaboração da apresentação final Critério de aceitação: A apresentação sintetiza os objetivos, arquitetura, metodologia, principais resultados e conclusões do projeto.

5.2. Ganhos esperados

Espera-se que o projeto demonstre a viabilidade da implementação, em FPGA, de uma arquitetura capaz de processar tráfego TCP/IP e realizar a extração de características baseadas em fluxo diretamente em hardware. A solução deverá permitir que informações associadas aos fluxos sejam identificadas, armazenadas e atualizadas à medida que os pacotes são recebidos, reduzindo a necessidade de armazenamento integral do tráfego para processamento posterior.

Também se espera obter uma arquitetura modular e extensível, composta por blocos com responsabilidades bem definidas, facilitando sua evolução. A estrutura proposta poderá ser posteriormente ampliada com novas features, maior capacidade de armazenamento de fluxos, mecanismos mais eficientes de tratamento de colisões e estratégias de processamento com maior paralelismo, como arquiteturas em pipeline.

O desenvolvimento de um ambiente de verificação baseado em arquivos PCAP, tráfego sintético e modelo de referência em software deverá fornecer uma estrutura reproduzível para a validação funcional dos módulos e da arquitetura integrada, tanto em simulação quanto na implementação em FPGA. A utilização de um dataset benchmark deverá ainda permitir avaliar o funcionamento do sistema com tráfego representativo de cenários reais de rede.

Por fim, a geração das features `packet_count`, `byte_count` e `duration` deverá estabelecer uma base para futuras extensões do projeto, incluindo a extração de características mais complexas e a utilização dos vetores produzidos como entrada para modelos de aprendizado de máquina aplicados à detecção de intrusões em redes.

6. Cronograma de Execução

6.1. Cronograma com Detalhamento Descritivo de Atividades

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
Extração de Metadados	M1.1	Especificação das interfaces de todos os módulos	Evita incompatibilidades na comunicação e na integração entre os módulos.	Especificação documentada	1	Sem. 1	Sem. 3
	M1.2	Especificação da lógica de funcionamento de todos os módulos	Define o comportamento esperado e orienta a implementação dos módulos.	Especificação documentada	1	Sem. 1	Sem. 3
	M1.3	Especificação dos protocolos suportados e do algoritmo de hash	Delimita o escopo do Parser e define a geração do índice para consulta à Flow Table.	Especificação documentada	1	Sem. 1	Sem. 3
	M1.4	Implementação da recepção dos pacotes pelo Parser	Garante o recebimento correto dos pacotes pela interface de entrada.	Módulo iniciado	1	Sem. 3	Sem. 4
	M1.5	Implementação da FSM com a lógica principal do Parser	Controla a interpretação sequencial dos cabeçalhos e a extração dos metadados.	Funcionalidade implementada	1	Sem. 4	Sem. 6
	M1.6	Implementação do tratamento dos pacotes fora do escopo pelo Parser	Impede que pacotes não suportados produzam metadados inválidos.	Funcionalidade implementada	1	Sem. 6	Sem. 7

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
	M1.7	Implementação do módulo Flow Identification Unit	Permite identificar o fluxo correspondente a cada pacote por meio da 5-tupla.	Módulo implementado	1	Sem. 3	Sem. 5
	M1.8	Implementação do módulo Hash Unit	Gera o índice utilizado para a consulta inicial à Flow Table.	Módulo implementado	1	Sem. 3	Sem. 5
	M1.9	Integração e validação do Bloco Extração de Metadados	Comprova o funcionamento conjunto dos módulos de Extração de Metadados.	Bloco validado	1	Sem. 7	Sem. 9
Extração de Features	M2.1	Especificação das interfaces de todos os módulos e da estrutura da Flow Table	Define como os módulos irão trocar dados e informações, evitando incompatibilidades durante a integração.	Especificação documentada	1	Sem. 1	Sem. 3
	M2.2	Especificação da lógica de funcionamento de todos os módulos	Define as sequências de operação, estados e mecanismos de controle necessários para implementação do bloco.	Especificação documentada	1	Sem. 1	Sem. 3
	M2.3	Implementação do módulo Flow Table	Desenvolve a estrutura responsável pelo armazenamento dos registros e estados dos fluxos.	Módulo implementado	1	Sem. 3	Sem. 5

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
	M2.4	Implementação da recepção dos metadados e solicitação de LOOKUP pelo Controller	Permite iniciar o processamento de cada pacote e realizar a consulta da entrada correspondente na Flow Table.	Módulo iniciado	1	Sem. 3	Sem. 4
	M2.5	Implementação da classificação do resultado do LOOKUP em HIT, MISS e COLLISION	Permite determinar corretamente a operação necessária após a consulta à Flow Table.	Funcionalidade implementada	1	Sem. 4	Sem. 5
	M2.6	Implementação do tratamento de MISS e COLLISION	Permite inserir novos fluxos em posições livres e tratar colisões conforme a estratégia definida para a primeira PoC.	Funcionalidade implementada	1	Sem. 5	Sem. 6
	M2.7	Implementação do tratamento de HIT.	Permite identificar fluxos existentes e encaminhar seus estados para atualização.	Funcionalidade implementada	1	Sem. 6	Sem. 7
	M2.8	Implementação do módulo Feature Update Unit	Desenvolve a lógica responsável pela atualização incremental das informações e features dos fluxos.	Módulo implementado	1	Sem. 3	Sem. 5
	M2.9	Implementação do controle de fluxo e backpressure	Garante o controle correto do recebimento de novos pacotes durante operações pendentes de leitura, escrita e atualização.	Funcionalidade implementada	1	Sem. 7	Sem. 8

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
	M2.10	Integração dos módulos Controller e Flow Table para solicitações de LOOKUP e INSERT	Valida o funcionamento conjunto da consulta e inserção de novos registros de fluxo.	Integração validada	1	Sem. 8	Sem. 9
	M2.11	Integração dos módulos Controller, Feature Update Unit e Flow Table para solicitações de UPDATE	Permite validar o ciclo completo de leitura, atualização e escrita de um fluxo existente.	Integração validada	1	Sem. 9	Sem. 10
	M2.12	Implementação do módulo Flow Finalizer	Desenvolve a geração das features finais a partir dos estados acumulados dos fluxos.	Módulo implementado	1	Sem. 5	Sem. 7
	M2.13	Integração dos módulos Controller e Flow Finalizer	Permite realizar o processo de finalização e o envio dos registros válidos para geração dos vetores de features.	Integração validada	1	Sem. 10	Sem. 12
	M2.14	Integração e validação do Bloco Extração de Features	Verifica o funcionamento integrado de todas as etapas, desde a recepção dos metadados e consulta à Flow Table até a atualização dos estados e geração das features.	Bloco Validado	1	Sem. 12	Sem. 14
Ambiente de Verificação	M3.1	Especificação da estratégia de verificação	Define o fluxo completo de verificação, desde os arquivos PCAP e geração dos estímulos	Estratégia de verificação documentada	1	Sem. 1	Sem. 3

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
			até a captura e comparação das saídas do DUT				
	M3.2	Especificação das interfaces de entrada e saída do ambiente de simulação	Define os sinais, protocolo de transferência e comportamento da interface utilizada para aplicação dos pacotes ao DUT, considerando a compatibilidade com IP MAC e AXI-Stream.	Interface especificada	1	Sem. 1	Sem. 3
	M3.3	Especificação do formato de estímulos	Define o formato dos arquivos utilizados incluindo dados dos pacotes, delimitação entre pacotes, bytes válidos e informações de timestamp	Formato de estímulos especificado	1	Sem. 1	Sem. 3
	M3.4	Geração de pacotes sintéticos	Desenvolve mecanismos para criação de tráfego sintético controlado, permitindo gerar diferentes combinações de fluxos, tamanhos de pacotes, timestamps e situações de teste.	Conjunto de PCAPs sintéticos	1	Sem. 3	Sem. 4
	M3.5	Desenvolvimento do conversor de PCAP para arquivo de estímulos em Python	Lê arquivos PCAP e os converte para o formato de estímulos	Conversor implementado e validado	1	Sem. 4	Sem. 6

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
			utilizado pelo ambiente de simulação.				
	M3.6	Implementação do testbench parcial para extração de metadados	Desenvolve um testbench para inicialização do DUT, leitura dos estímulos e aplicação dos pacotes, permitindo observar e registrar os metadados	Testbench implementado	1	Sem. 5	Sem. 8
	M3.7	Validação do bloco de Extração de Metadados	Executa casos de teste para verificar se os campos extraídos e os metadados produzidos correspondem aos valores esperados.	Bloco validado	1	Sem. 8	Sem. 9
	M3.8	Implementação do testbench parcial para extração de features	Desenvolve um testbench para aplicar sequências conhecidas de flow_key, hash_index, packet_length e timestamp, permitindo verificar as operações de LOOKUP, INSERT, UPDATE e finalização.	Testbench implementado	1	Sem. 9	Sem. 13
	M3.9	Validação do bloco de Extração de Features	Verifica o comportamento da Flow Table e dos módulos associados para diferentes sequências de pacotes, incluindo situações de HIT, MISS e	Bloco validado	1	Sem. 13	Sem. 14

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
			COLLISION e a geração das features finais.				
	M3.10	Definição e criação dos casos de teste com pacotes sintéticos	Estrutura casos de teste representativos, incluindo fluxos de um e múltiplos pacotes, fluxos intercalados, diferentes comprimentos, pacotes fora do escopo e situações de colisão e descarte.	Conjunto de casos de teste	1	Sem. 9	Sem. 10
	M3.11	Definição do método de associação das saídas do sistema	Define como cada vetor de features produzido pelo DUT será associado ao fluxo correspondente, permitindo relacionar os resultados obtidos em simulação aos valores calculados pelo modelo de referência.	Método definido	1	Sem. 10	Sem. 11
	M3.12	Implementação do modelo de referência em software	Desenvolve um modelo de referência capaz de processar os mesmos pacotes utilizados pelo DUT e calcular os valores esperados de packet_count, byte_count e duration para cada fluxo.	Modelo de referência implementado	1	Sem. 11	Sem. 13

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
	M3.13	Implementação do testbench completo para validação da arquitetura	Integra os mecanismos de aplicação dos estímulos, execução da arquitetura completa, captura das saídas e obtenção dos resultados necessários para comparação com o modelo de referência.	Testbench completo implementado	1	Sem. 13	Sem. 14
	M3.14	Validação da arquitetura completa	Executa o fluxo completo de processamento, desde a aplicação dos pacotes na entrada até a geração dos vetores de features, comparando os resultados do DUT com o modelo de referência.	Arquitetura validada	1	Sem. 17	Sem. 18
	M3.15	Escolha de um dataset benchmark de ameaças para testes	Seleciona um dataset público contendo tráfego benigno e malicioso em formato PCAP, considerando representatividade, compatibilidade com os protocolos suportados e adequação ao escopo da primeira PoC.	Dataset benchmark selecionado	1	Sem. 4	Sem. 6

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
	M3.16	Preparação/adaptação do dataset benchmark para o ambiente de verificação	Prepara um subconjunto representativo do dataset, considerando a compatibilidade com os protocolos suportados e sua conversão para o formato de estímulos utilizado pelo ambiente de simulação.	Dataset preparado	1	Sem. 6	Sem. 8
	M3.17	Verificação do sistema completo com dataset benchmark	Processa um subconjunto representativo do dataset no ambiente de simulação e compara os vetores produzidos pelo DUT com os resultados calculados pelo modelo de referência.	Verificação com benchmark concluída	1	Sem. 18	Sem. 19
	M3.18	Implementação do circuito em FPGA	Integra e sintetiza os módulos desenvolvidos na FPGA selecionada, verificando a implementação e registrando a utilização dos recursos e a frequência de operação obtida.	Circuito implementado em FPGA	1	Sem. 18	Sem. 19

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
	M3.19	Implementação de mecanismo externo para envio de estímulos e captura de resultados em FPGA	Desenvolve o mecanismo necessário para fornecer os pacotes à arquitetura implementada na FPGA e recuperar externamente os vetores de features produzidos pelo sistema.	Mecanismo implementado	1	Sem. 19	Sem. 20
	M3.20	Validação da arquitetura implementada em FPGA	Executa pacotes conhecidos e um subconjunto representativo do dataset benchmark na FPGA, comparando os resultados obtidos com aqueles produzidos em simulação e pelo modelo de referência.	Arquitetura validada em FPGA	1	Sem. 20	Sem. 21
Documentação	M4.1	Documentação da arquitetura do sistema	Consolida a arquitetura desenvolvida, incluindo os módulos, interfaces, fluxos de dados e principais decisões de projeto, garantindo a rastreabilidade da implementação.	Arquitetura documentada	1	Sem. 17	Sem. 22
	M4.2	Documentação dos procedimentos de verificação	Registra os procedimentos utilizados para geração e aplicação dos estímulos, execução dos testbenches,	Procedimentos documentados	1	Sem. 17	Sem. 22

Atividade	Meta	Descrição	Justificativa	Indicador Físico		Previsão de execução	
				Unidade de medida	Quantidade	Início	Fim
			comparação dos resultados e validação em simulação e FPGA.				
	M4.3	Registro dos resultados de simulação	Consolida os resultados obtidos durante os testes dos blocos e da arquitetura completa, permitindo documentar o atendimento aos requisitos definidos.	Resultados documentados	1	Sem. 17	Sem. 22
	M4.4	Elaboração do relatório final do projeto	Reúne a fundamentação, metodologia, desenvolvimento, procedimentos de verificação e resultados obtidos durante o projeto.	Relatório redigido	1	Sem. 17	Sem. 22
	M4.5	Elaboração da apresentação final	Sintetiza os objetivos, arquitetura, metodologia, resultados e conclusões do projeto para apresentação do trabalho desenvolvido.	Apresentação concluída	1	Sem. 23	Sem. 24

6.2. Visão Temporal do Cronograma de Execução

[illegible]

[illegible]

[illegible]

Meta	Indicador físico		Semana																							
	Unidade de medida	Quantidade	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
M3.9	Bloco validado	1														x										
M3.10	Conjunto de casos de teste criado	1										x														
M3.11	Método definido	1											x													
M3.12	Modelo de referência implementado	1													x											
M3.13	Testbench completo implementado															x										
M3.14	Arquitetura completa validada																			x						
M3.15	Dataset benchmark selecionado							x																		
M3.16	Dataset preparado									x																
M3.17	Verificação com benchmark concluída																				x					

Meta	Indicador físico		Semana																							
	Unidade de medida	Quantidade	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
M3.18	Circuito implementado	1																			x					
M3.19	Mecanismo implementado	1																				x				
M3.20	Arquitetura validada em FPGA	1																					x			
M4.1	Arquitetura documentada																									
M4.2	Procedimentos documentados																									
M4.3	Resultados documentados																									
M4.4	Relatório redigido																									
M4.5	Apresentação concluída																									

Nota: Semanas preenchidas com sombreamento em cinza indicam o período de execução da meta, enquanto as células assinaladas com "x" indicam a semana de entrega de um indicador físico. Semanas preenchidas com sombreamento em vermelho indicam o período de pausa.

7. Planejamento das Ações

7.1. Gerenciamento de Mudanças no Escopo

As mudanças no escopo deverão ser registradas e avaliadas pela equipe, considerando sua justificativa, impacto no cronograma, nos módulos envolvidos e nos entregáveis previstos. Alterações que afetem as interfaces entre os blocos de Extração de Metadados, Extração de Features e Ambiente de Verificação deverão ser discutidas antes da implementação.

A primeira prova de conceito terá como prioridade o processamento de pacotes Ethernet, IPv4 e TCP/UDP, a identificação de uniflows e a extração das features ``packet_count``, ``byte_count`` e ``duration``. Funcionalidades adicionais, como biflows, tratamento avançado de colisões, timeout de fluxos e features estatísticas mais complexas, somente serão incorporadas após a validação da arquitetura básica e desde que não comprometam os marcos principais do projeto.

7.2. Mapeamento da Execução e Interdependências

A execução do projeto possui dependências entre as diferentes frentes de desenvolvimento. A especificação das interfaces e da lógica dos blocos de Extração de Metadados e Extração de Features deverá ser compatível, pois os valores gerados pelo primeiro bloco, incluindo ``flow_key``, ``hash_index``, ``packet_length`` e ``timestamp``, serão utilizados pelo Controller e pela Flow Table.

O Ambiente de Verificação será desenvolvido de forma progressiva e acompanhará a implementação dos blocos de hardware, inicialmente por meio de estímulos sintéticos e testbenches parciais e, posteriormente, pela validação da arquitetura completa com casos de teste sintéticos e um dataset benchmark. A integração final dependerá da conclusão e validação dos blocos de Extração de Metadados e Extração de Features, permitindo verificar o caminho completo desde a entrada dos pacotes até a geração dos vetores de features.

7.3. Controle de Alterações no Cronograma

O cronograma será acompanhado periodicamente pela equipe, considerando o andamento das metas e as dependências entre as atividades. Eventuais desvios deverão ser registrados e avaliados quanto ao impacto nas metas posteriores, principalmente nas atividades de integração e validação.

Sempre que uma atividade sofrer atraso, serão priorizados os entregáveis necessários para a validação da primeira prova de conceito. Funcionalidades não essenciais poderão ser postergadas, preservando como prioridade a implementação e validação do fluxo completo de processamento: recepção dos pacotes, extração dos metadados, identificação e armazenamento dos fluxos, atualização das features e geração do vetor final. As últimas semanas do período de execução serão destinadas principalmente à integração, verificação, implementação em FPGA e consolidação dos resultados.

7.4. Mapeamento de Riscos e Plano de Ação

Os principais riscos identificados para o projeto e as respectivas ações previstas são:

- **Incompatibilidade entre os blocos:** alterações nas interfaces ou larguras dos sinais podem comprometer a integração. Como ação, as interfaces e os protocolos de comunicação deverão ser definidos e documentados antes da implementação.

- **Erros na identificação ou associação dos fluxos:** falhas na geração da flow_key ou do hash_index podem associar pacotes ao fluxo incorreto. Serão utilizados casos de teste controlados e um modelo de referência para verificar os resultados.
- **Colisões na Flow Table:** diferentes fluxos podem ser direcionados à mesma posição. Na primeira PoC, a ocorrência será sinalizada e o novo fluxo não será inserido; mecanismos mais avançados poderão ser avaliados posteriormente.
- **Inconsistências na atualização das features:** pacotes consecutivos podem acessar um registro antes da conclusão da atualização anterior. Inicialmente, será utilizado controle por valid/ready e backpressure para garantir a conclusão das operações antes do processamento seguinte.
- **Dificuldades na finalização dos fluxos:** mecanismos baseados em timeout ou eventos de protocolo aumentam a complexidade da arquitetura. Na primeira implementação, a finalização será realizada ao término do arquivo PCAP.
- **Divergência entre hardware e resultados de referência:** os valores produzidos pela arquitetura podem não corresponder aos valores esperados. Serão utilizados testbenches, dados sintéticos, dataset benchmark e modelo de referência em software para identificar e analisar eventuais divergências.
- **Limitações de recursos ou timing na FPGA:** o aumento do número de entradas da Flow Table ou das features pode elevar a utilização de memória e lógica. A implementação será realizada de forma progressiva, iniciando com uma estrutura reduzida e ampliando-a após a validação funcional.
- **Atrasos ou mudanças na disponibilidade dos integrantes:** caso algum membro deixe de participar ou passe a ter indisponibilidade significativa durante a execução, as atividades deverão ser redistribuídas entre os integrantes restantes, priorizando os módulos e entregáveis essenciais para a conclusão da primeira prova de conceito.